

C3M1: Peer Reviewed Assignment

Outline:

The objectives for this assignment:

- 1. Apply Binomial regression methods to real data.
- 2. Understand how to analyze and interpret binomial regression models.
- 3. Flex our math skills by determining whether certain distributions are members of the exponential family.

General tips:

- 1. Read the questions carefully to understand what is being asked.
- 2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

In [3]:

```
# Load required libraries
library(tidyverse)
library(dplyr)
library(magrittr)
```

Attaching packages

tidyverse 1.3.0

✓ ggplot2 3.3.0

✓ purrr 0.3.4

✓ tibble 3.2.1

✓ dplyr 1.1.2

✓ tidyr 1.0.2

✓ stringr 1.4.0

✓ readr 1.3.1

✓ forcats 0.5.0

Conflicts

tidyverse_conflicts()

✗ dplyr::filter()

masks stats::filter()

✗ dplyr::lag()

masks stats::lag()

Attaching package: ‘magrittr’

The following object is masked from ‘package:purrr’:

set_names

The following object is masked from ‘package:tidyr’:

extract

Problem 1: Binomial (Logistic) Regression

The National Institute of Diabetes and Digestive and Kidney Diseases conducted a study of 768 adult female Pima Indians living near Phoenix, AZ. The purpose of the study was to investigate the factors related to diabetes.

Before we analyze these data, we should note that some have raised ethical issues with its collection and popularity in the statistics and data science community. We should think seriously about these concerns. For example, Maya Iskandarani wrote a brief [piece \(https://researchblog.duke.edu/2016/10/24/diabetes-and-privacy-meet-big-data/\)](https://researchblog.duke.edu/2016/10/24/diabetes-and-privacy-meet-big-data/) on consent and privacy concerns raised by this dataset. After you familiarize yourself with the data, we'll then turn to these ethical concerns.

First, we'll use these data to get some practice with GLM and Logistic regression.

In [4]:

```
# Load the data
pima = read.csv("pima.txt", sep="\t")
# Here's a description of the data: https://rdrr.io/cran/faraway/man/pima.html
head(pima)
```

A data.frame: 6 × 9

	pregnant	glucose	diastolic	triceps	insulin	bmi	diabetes	age	test
	<int>	<int>	<int>	<int>	<int>	<dbl>	<dbl>	<int>	<int>
1	6	148	72	35	0	33.6	0.627	50	1
2	1	85	66	29	0	26.6	0.351	31	0
3	8	183	64	0	0	23.3	0.672	32	1
4	1	89	66	23	94	28.1	0.167	21	0
5	0	137	40	35	168	43.1	2.288	33	1
6	5	116	74	0	0	25.6	0.201	30	0

1. (a) Data Cleaning? What about Data Scrubbing? Data Sterilizing?

This is a real data set, which means that there's likely going to be gaps and missing values in the data. Before doing any modeling, we should inspect the data and clean it if necessary.

Perform simple graphical and numerical summaries of the data. Pay attention for missing or nonsensical values. Can you find any obvious irregularities? If so, take appropriate steps to correct these problems. In the markdown cell, specify what cleaning you did and why you did it.

Finally, split your data into training and test sets. Let the training set contain 80% of the rows and the test set contain the remaining 20%.

```
In [5]: summary(pima)
sum(is.na(pima))
# glucose level of zero makes no sense
# blood pressure of zero makes no sense (unless person is dead)
# triceps skin fold thickness of zero looks like NA in histogram
# insulin of zero makes no sense
# BMI of zero makes no sense

hist(pima[, "triceps"], breaks = 50)

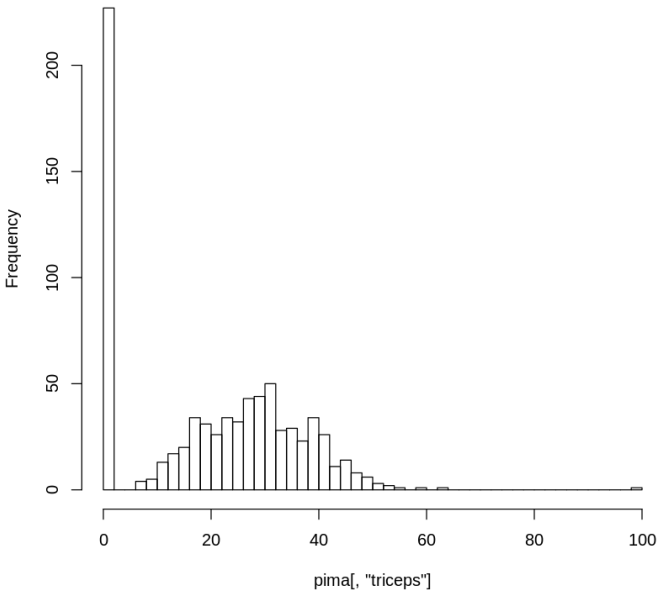
INDEX <- (pima$glucose == 0 | pima$diastolic == 0 | pima$triceps == 0 | pima$insulin == 0 | pima$bmi == 0)
pima <- pima[!(INDEX),]
hist(pima[,4])

set.seed(77)
train.index <- sample(1:nrow(pima), ceiling(nrow(pima)*0.8), )
pima.train <- pima[train.index,]
pima.test <- pima[-train.index,]
```

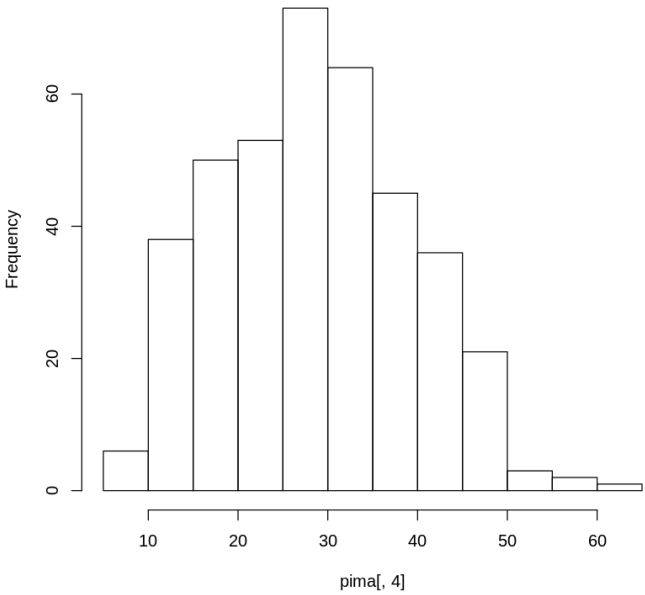
pregnant	glucose	diastolic	triceps
Min. : 0.000	Min. : 0.0	Min. : 0.00	Min. : 0.00
1st Qu.: 1.000	1st Qu.: 99.0	1st Qu.: 62.00	1st Qu.: 0.00
Median : 3.000	Median :117.0	Median : 72.00	Median :23.00
Mean : 3.845	Mean :120.9	Mean : 69.11	Mean :20.54
3rd Qu.: 6.000	3rd Qu.:140.2	3rd Qu.: 80.00	3rd Qu.:32.00
Max. :17.000	Max. :199.0	Max. :122.00	Max. :99.00
insulin	bmi	diabetes	age
Min. : 0.0	Min. : 0.00	Min. :0.0780	Min. :21.00
1st Qu.: 0.0	1st Qu.:27.30	1st Qu.:0.2437	1st Qu.:24.00
Median : 30.5	Median :32.00	Median :0.3725	Median :29.00
Mean : 79.8	Mean :31.99	Mean :0.4719	Mean :33.24
3rd Qu.:127.2	3rd Qu.:36.60	3rd Qu.:0.6262	3rd Qu.:41.00
Max. :846.0	Max. :67.10	Max. :2.4200	Max. :81.00
test			
Min. :0.000			
1st Qu.:0.000			
Median :0.000			
Mean :0.349			
3rd Qu.:1.000			
Max. :1.000			

0

Histogram of pima[, "triceps"]



Histogram of pima[, 4]



All rows with at least one zero value measurement for the variables glucose, diastolic (blood pressure), triceps, insulin and bmi were discarded. For these variables a value of zero makes barely any sense and therefore treated as NA.

1. (b) Initial GLM modelling

Our data is clean and we're ready to fit! What kind of model should we use to fit these data? Notice that the `test` variable is either 0 or 1, for whether the individual tested positive for diabetes. Because `test` is binary, we should use logistic regression (which is a kind of binomial regression).

Fit a model with `test` as the response and all the other variables as predictors. Can you tell whether this model fits the data?

```
In [6]: glmmod.pima <- glm(test ~ ., data = pima.train)
summary(glmmod.pima)
summary(glmmod.pima)$coef[-1,4] %>%
  p.adjust("fdr")

Call:
glm(formula = test ~ ., data = pima.train)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.03684  -0.23928  -0.04701   0.25111   1.04864

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.1425829   0.1567327  -7.290 2.68e-12 ***
pregnant     0.0235598   0.0093391   2.523  0.0122 *
glucose      0.0069566   0.0009022   7.711 1.78e-13 ***
diastolic    0.0013725   0.0020005   0.686  0.4932
triceps      0.0019117   0.0027873   0.686  0.4933
insulin     -0.0002357   0.0002294  -1.027  0.3051
bmi          0.0088921   0.0044731   1.988  0.0477 *
diabetes     0.1370829   0.0694992   1.972  0.0495 *
age          0.0023824   0.0031843   0.748  0.4549
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.1459667)

    Null deviance: 70.854  on 313  degrees of freedom
Residual deviance: 44.520  on 305  degrees of freedom
AIC: 297.71

Number of Fisher Scoring iterations: 2

pregnant: 0.0486164696474529 glucose: 1.42530860664819e-12 diastolic: 0.493325758474275 triceps: 0.493325758474275 insulin: 0.48814511841522 bmi:
0.0989253305394753 diabetes: 0.0989253305394753 age: 0.493325758474275
```

Looking at the results from the t-test, 4 out of 8 of the potential predictors show statistically significant p values at the alpha = 0.05 level. Well, the variables `pregnant`, `bmi` and `diabetes` are associated with p values rather close to the alpha threshold of 0.05, and when correcting for multiple testing only `glucose` (and `pregnant`) survive as robust features. The % of deviance explained by the model is ~ 37 %. However, no overall significance test is directly outputted by the `summary()` function. Overall, for now it appears that the model can explain about a third of the variability in the response variable.

1. (c) Remember Bayes

A quick analytical interlude.

Is diastolic blood pressure significant in the regression model? Do women who test positive have higher diastolic blood pressures? Explain the distinction between the two questions and discuss why the answers are only apparently contradictory.

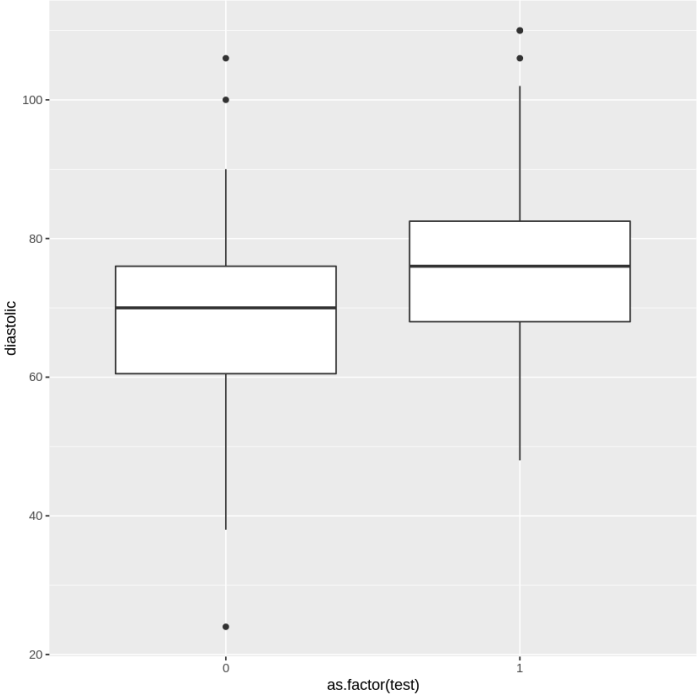
```
In [7]: print("p-values associated with diastolic blood pressure from t-tests: ")
print(paste0("for above logistic regression model: ", round(summary(glmmod.pima)$coef[4,4],2)))

print(paste0("when diastolic blood pressure is grouped by test variable: ",
             round(t.test(pima.train$diastolic[pima.train$test == 0],
                           pima.train$diastolic[pima.train$test == 1])$p.value,5)
             ))

print(paste0("for logistic regression model with only diastolic as predictor: ",
             round(summary(glm(test ~ diastolic, data = pima.train))$coef[2,4],5)
             ))

pima.train %>%
  ggplot(aes(x = as.factor(test), y = diastolic)) +
  geom_boxplot()
```

[1] "p-values associated with diastolic blood pressure from t-tests: "
[1] "for above logistic regression model: 0.49"
[1] "when diastolic blood pressure is grouped by test variable: 5e-05"
[1] "for logistic regression model with only diastolic as predictor: 3e-05"



The differences most likely result from other variables. When we merely consider diastolic blood pressure (and every other variable is left out), a logistic regression model suggests that differences in diastolic blood pressure are associated with differences in the probabilities of testing positive for diabetes. When we include the other variables into the model and thus condition on them, the apparent association between diastolic blood pressure and probability of testing positive for diabetes is "explained away" by the other variables. This is essentially why the two questions asked are distinct.

1. (d) GLM Interpretation

We've seen so many regression summaries up to this point, how is this one different from all the others? Well, to really understand any model, it can be helpful to loop back and plug the fitted results back into the model's mathematical form.

Explicitly write out the equation for the binomial regression model that you fit in (b). Then, in words, explain how a 1 unit change of glucose affects test, assuming all other predictors are held constant.

```
In [8]: # Your Code Here
summary(glmmod.pima)
```

Call:
glm(formula = test ~ ., data = pima.train)

Deviance Residuals:
 Min 1Q Median 3Q Max
-1.03684 -0.23928 -0.04701 0.25111 1.04864

Coefficients:
 Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.1425829 0.1567327 -7.290 2.68e-12 ***
pregnant 0.0235598 0.0093391 2.523 0.0122 *
glucose 0.0069566 0.0009022 7.711 1.78e-13 ***
diastolic 0.0013725 0.0020005 0.686 0.4932
triceps 0.0019117 0.0027873 0.686 0.4933
insulin -0.0002357 0.0002294 -1.027 0.3051
bmi 0.0088921 0.0044731 1.988 0.0477 *
diabetes 0.1370829 0.0694992 1.972 0.0495 *
age 0.0023824 0.0031843 0.748 0.4549

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.1459667)

 Null deviance: 70.854 on 313 degrees of freedom
Residual deviance: 44.520 on 305 degrees of freedom
AIC: 297.71

Number of Fisher Scoring iterations: 2

$$\ln(\frac{p_i}{1-p_i}) = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \beta_3 x_{i,3} + \beta_4 x_{i,4}$$

$$+ \beta_5 x_{i,5} + \beta_6 x_{i,6} + \beta_7 x_{i,7} + \beta_8 x_{i,8}$$

Where:

β_0 : *intercept*

β_1 : *pregnant*

β_2 : *glucose*

β_3 : *diastolic*

β_4 : *triceps*

β_5 : *insulin*

β_6 : *bmi*

β_7 : *diabetes*

β_8 : *age*

A one unit increase in glucose would result in an additive increase the log odds of receiving a positive test result by 0.007 (holding everything else constant). Put differently, a one unit increase in glucose would result in a multiplicative change by $e^{0.007}$ in the odds of receiving a positive test result.

1. (e) GLM Prediction

One of the downsides of Logistic Regression is that there isn't an easy way of evaluating the goodness of fit of the model without predicting on new data. But, if we have more data to test with, then there are many methods of evaluation to use. One of the best tools are confusion matrices, which (despite the name) are actually not that hard to understand.

A confusion matrix compares the predicted outcomes of a Logistic Regression Model (or any classification model) with the actual classifications. For binary classification, it is a 2×2 matrix where the rows are the models' predicted outcome and the columns are the actual classifications. An example is displayed below.

	True	False
1	103	37
0	55	64

In the example, we know the following information:

- The [1,1] cell is the number of datapoints that were correctly predicted to be 1. The value (103) is the number of True Positives (TP).
- The [2,2] cell is the number of datapoints that were correctly predicted to be 0. The value is the number of True Negatives (TN).
- The [1, 2] cell is the number of datapoints that were predicted to be 1 but where actually 0. This is the number of False Positives (FP), also called Type I error. In the context of our diabetes dataset, this would mean our model predicted that the person would have diabetes, but they actually did not.
- The [2, 1] cell is the number of datapoints that were predicted to be 0 but where actually 1. This is the number of False Negatives (FN), also called Type 2 error. In the context of our diabetes dataset, this would mean our model predicted that the person would not have diabetes, but they actually did have diabetes.

Use your model to predict the outcomes of the test set. Then construct a confusion matrix for these predictions and display the results.

In [9]:

```
# Your Code Here
pred <- predict(glmmod.pima, pima.test)
pred.binary <- ifelse(pred > 0.5, 1, 0)

conf.mat <- table(predicted = pred.binary, observed = pima.test$test)
conf.mat
```

observed
predicted 0 1
 0 49 12
 1 7 10

1. (f) Evaluation Statistics

Using the four values from the confusion matrix, we can construct evaluation statistics to get a numerical approximation for our model's performance. Spend some time researching accuracy, precision, recall and F score.

Calculate these values for your model's predictions on the test set. Clearly display your results. How well do you think your model fits the data?

```
In [10]: # Your Code Here
TP <- conf.mat[2,2]
TN <- conf.mat[1,1]
FP <- conf.mat[2,1]
FN <- conf.mat[1,2]

accuracy <- function(TP, TN, FP, FN) (TP+TN)/(TP+TN+FP+FN)
precision <- function(TP, FP) TP/(TP+FP)
recall <- function(TP, FN) TP/(TP+FN)
F1 <- function(precision, recall) (precision*recall)/(precision+recall)

accuracy.pima <- accuracy(TP, TN, FP, FN)
precision.pima <- precision(TP, FP)
recall.pima <- recall(TP, FN)
F1.pima <- F1(precision.pima, recall.pima)

print(paste("Accuracy:", round(accuracy.pima,3)))
print(paste("Precision:", round(precision.pima,3)))
print(paste("Recall:", round(recall.pima,3)))
print(paste("F1:", round(F1.pima,3)))

library(pROC)
# Compute the ROC curves
roc_model <- roc(as.numeric(pima.test$test), as.numeric(pred.binary))
roc_null <- roc(as.numeric(pima.test$test), rep(0, nrow(pima.test)))

# Compare the AUCs
roc.test(roc_model, roc_null)
```

[1] "Accuracy: 0.756"
[1] "Precision: 0.588"
[1] "Recall: 0.455"
[1] "F1: 0.256"

Type 'citation("pROC")' for a citation.

Attaching package: ‘pROC’

The following objects are masked from ‘package:stats’:

cov, smooth, var

Setting levels: control = 0, case = 1

Setting direction: controls < cases

Setting levels: control = 0, case = 1

Setting direction: controls < cases

DeLong's test for two correlated ROC curves

data: roc_model and roc_null
Z = 2.8058, p-value = 0.005019
alternative hypothesis: true difference in AUC is not equal to 0
sample estimates:
AUC of roc1 AUC of roc2
0.6647727 0.5000000

For this particular train/test split, the model performs moderate at best.

- 1. The no-information rate (i.e. the largest proportion of an observed class) is about 72 %, and thus almost the same as the model's accuracy of about 76 %.
- 2. However, when comparing the Receiver-operating characteristic curves for this model versus a null model (where the most abundant class is always predicted), a statistical difference is obtained. Note, however, that the AUC for this model is only about 0.66 (the null model's AUC is 0.5).

1. (g) Understanding Evaluation Statistics

Answer the following questions in the markdown cell below.

- 1. Give an example scenario for when accuracy would be a misleading evaluation statistic.
 - 2. Confusion matrices can also be used for non-binary classification problems. Describe what a confusion matrix would look like for a response with 3 levels.
 - 3. You'll have to take our word on the fact (or spend some time researching) that Type I error and Type II error are inversely related. That is, if a model is very good at detecting false positives, then it will be bad at detecting false negatives. In the case of our diabetes dataset, would you prefer a model that overestimates the Type 1 error or overestimates the Type II error. Justify your answer.
-
- 1. Suppose a binary classification problem, where one class makes up say 99 %. In this highly unbalanced case, a dumb null model would achieve an accuracy of 99 % by always choosing the dominant class. Nevertheless, in the biological sciences one can occasionally observe such curious claims, even in top tier journals. ;)
 - 2. In general, for a classification with k classes, the confusion matrix will be a k by k square matrix with the diagonal elements representing correct labels and the off-diagonal elements representing incorrect labels.
 - 3. In case of the diabetes dataset, most likely the aim of the model is a screening. This means that we aim for a high sensitivity, at the expend of a poor specificity. The idea is that the costs of a false positive prediction (i.e. claiming a person has/will have diabetes when he/she has not) are lower than the costs of missing a true positive (i.e. a person actually already diseased with diabetes without knowing it).

1. (h) Ethical Issues in Data Collection

Read Maya Iskandarani's [piece \(https://researchblog.duke.edu/2016/10/24/diabetes-and-privacy-meet-big-data/\)](https://researchblog.duke.edu/2016/10/24/diabetes-and-privacy-meet-big-data/), on consent and privacy concerns raised by this dataset. Summarize those concerns here.

The Pima Indian Diabetes Dataset (PIDD) raises significant privacy concerns due to the long-term and public accessibility of sensitive health information, such as blood pressure, BMI, and pregnancy history, collected from the Pima Native American tribe. The data, which has been publicly available in the UCI Machine Learning Repository for over two decades, is widely used for testing machine learning algorithms. Despite its value in advancing machine learning and diabetes research, the public availability of this data for over two decades poses ethical issues. These include the challenge of obtaining meaningful, "eternal" consent from participants, as it is impossible to predict how their data will be used decades into the future. This situation highlights the complexities and potential risks of handling medical data across generations.

Problem 2: Practicing those Math skills

One of the conditions of GLMs is that the "random component" of the data needs to come from the Exponential Family of Distributions. But how do we know if a distribution is in the Exponential Family? Well, we could look it up. Or we could be proper mathematicians and check the answer ourselves! Let's flex those math muscles.

2. (a) But it's in the name...

Show that $Y \sim exponential(\lambda)$, where λ is known, is a member of the exponential family.

The PDF for the exponential family of distributions is:

$$f(y|\theta, \phi) = exp\left(\frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi)\right)$$

The PDF for the exponential is:

$$f(y|\lambda) = P(Y = y|\lambda) = \lambda e^{-\lambda y} = e^{ln(\lambda) - \lambda y} = e^{y(-\lambda) + ln(\lambda)}$$

When we now set

$$\begin{aligned}\theta &= -\lambda \\ b(\theta) &= -ln(-\theta) \\ a(\phi) &= 1 \\ c(y, \phi) &= 0\end{aligned}$$

we can confirm that the exponential distribution belongs to the exponential family of distributions.

2. (b) Why can't plants do math? Because it gives them square roots!

Let $Y_i \sim exponential(\lambda)$ where $i \in \{1, \dots, n\}$. Then $Z = \sum_{i=1}^n Y_i \sim Gamma(n, \lambda)$. Show that Z is also a member of the exponential family.

The PDF for the exponential family of distributions is:

$$f(y|\theta, \phi) = exp\left(\frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi)\right)$$

The PDF for the gamma distribution is:

$$\begin{aligned}f(y|\alpha, \beta) &= \frac{\beta^\alpha}{\Gamma(\alpha)} y^{\alpha-1} exp(-\beta y) \\ &= exp(\alpha ln(\beta) - ln(\Gamma(\alpha)) + (\alpha - 1)ln(y) - \beta y) \\ &= exp(-\beta y + \alpha ln(\beta) + (\alpha - 1)ln(y) - ln(\Gamma(\alpha))) \\ &= exp\left(\frac{-\frac{\beta}{\alpha}y + ln(\beta)}{\frac{1}{\alpha}} + (\alpha - 1)ln(y) - ln(\Gamma(\alpha))\right) \\ &= exp\left(\frac{\frac{\beta}{\alpha}y - ln(\beta)}{-\frac{1}{\alpha}} + (\alpha - 1)ln(y) - ln(\Gamma(\alpha))\right)\end{aligned}$$

Now let's define $\theta = \frac{\beta}{\alpha}$ and $\phi = \frac{1}{\alpha}$. Therefore, $\beta = \theta\alpha = \frac{\theta}{\phi}$ and $ln(\beta) = ln(\theta) - ln(\phi)$

$$\begin{aligned}&= exp\left(\frac{y\theta - ln(\theta) + ln(\phi)}{-\phi} + (\frac{1}{\phi} - 1)ln(y) - ln(\Gamma(\frac{1}{\phi}))\right) \\ &= exp\left(\frac{y\theta - ln(\theta)}{-\phi} - \frac{ln(\phi)}{\phi} + (\frac{1}{\phi} - 1)ln(y) - ln(\Gamma(\frac{1}{\phi}))\right)\end{aligned}$$

When we now define $a(\phi) = -\phi$, $b(\theta) = ln(\theta)$ and $c(y, \phi) = -\frac{ln(\phi)}{\phi} + (\frac{1}{\phi} - 1)ln(y) - ln(\Gamma(\frac{1}{\phi}))$

we can confirm that the gamma distribution is a member of the exponential family of distributions.

```
In [11]: # As a numeric proof:

# the original gamma pdf:
gamma_ori <- function(y, alpha, beta) {
  (beta^alpha)/(gamma(alpha))*y^(alpha-1)*exp(-beta*y)
}

# the gamma pdf, in form of the exponential family of distributions:
gamma_expofam <- function(y, theta, phi) {
  exp((y*theta - log(theta))/(-phi) - log(phi)/(phi) + (1/phi-1)*log(y) - log(gamma(1/phi)) )
}

y <- 2; alpha <- 3; beta <- 2
gamma_ori(y, alpha, beta)

theta <- beta/alpha; phi <- 1/alpha
gamma_expofam(y, theta, phi)
```

0.293050222219747
0.293050222219747